# MyResume - Phase 1: ClickOps

## Overview

My Resume is an online, cloud-deployed version of my portfolio. This project focuses on the practical application of Microsoft Azure services and demonstrating foundational cloud architecture and deployment skills. This project is developed as the kickstart for my Cloud Computing journey.

Phase 1 of this project involves ClickOps, where after developing the different stacks locally, I proceeded into Azure and manually click on the different services for deployment of the application over the web.

This document will serve as the overview for MyResume Phase 1: ClickOps.

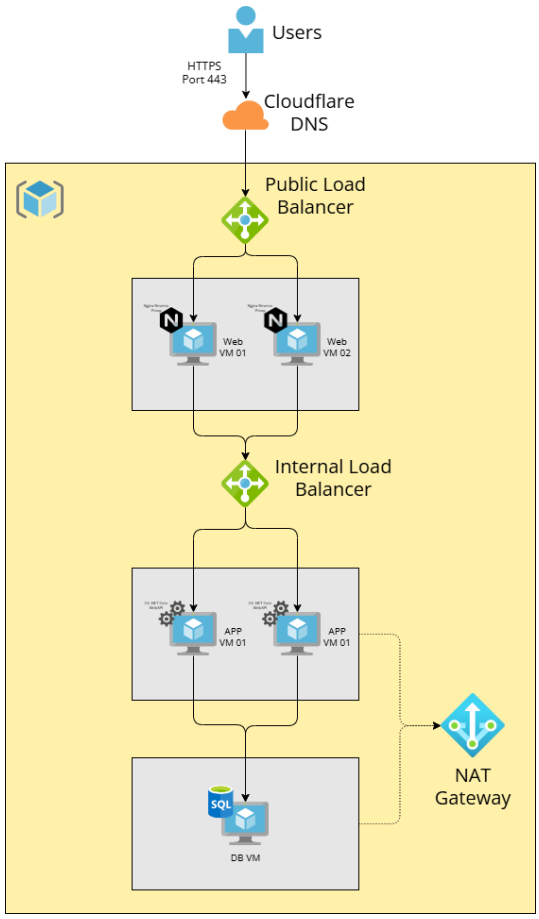
The Tech Stack:

- Frontend: React with Typescript (Hosted via Nginx in the Virtual Machine)
- Backend: C# .NET Core Web API (Ran as service with Kestrel)
- Database: Microsoft SQL Server

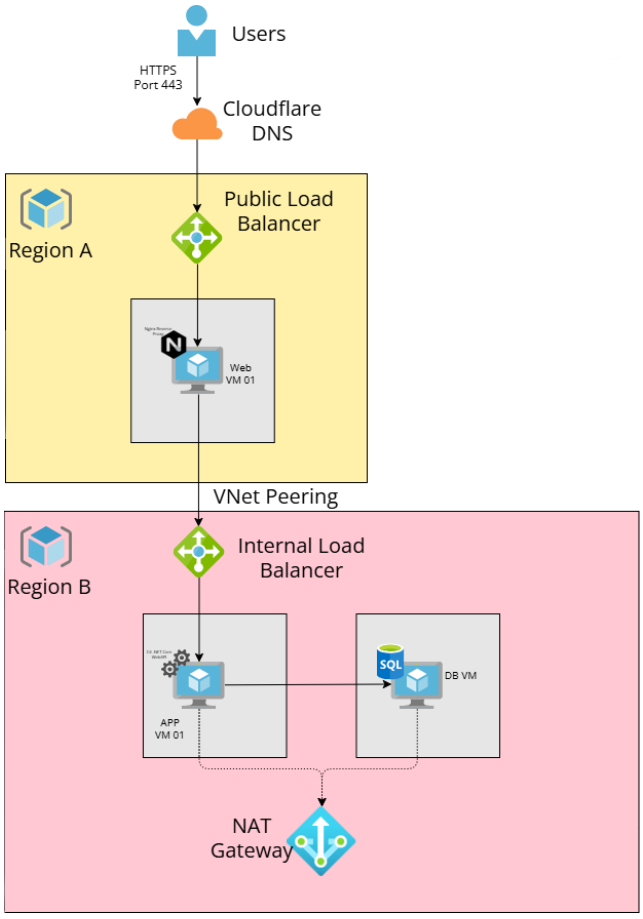
# Contents

# Architecture

The planned architecture was a fully redundant, single region deployment optimised for high availability and low internal latency:



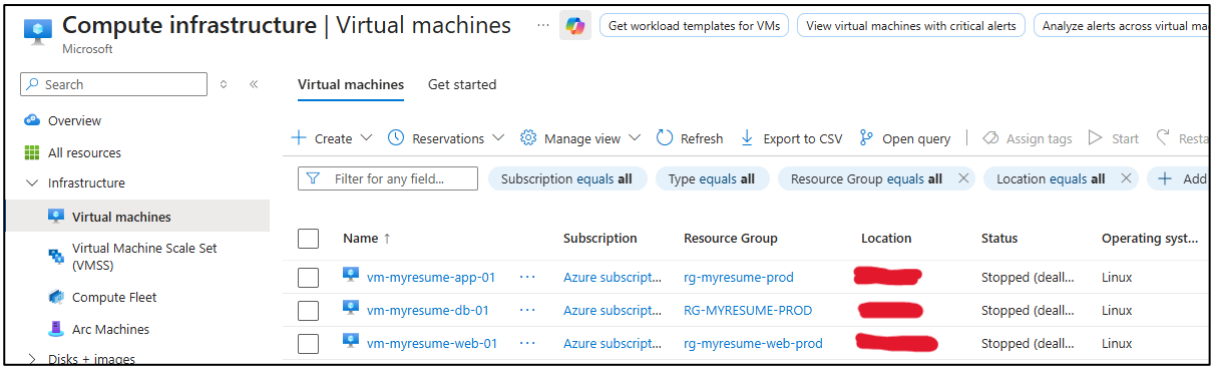

However, due to the limitations of Azure Free Trial, each region can only have 4 vCPUs. I had to modify the actual plan into a budget conscious optimisation designed to maximise Azure Free Trial while maintaining that multi-tier routing logic:



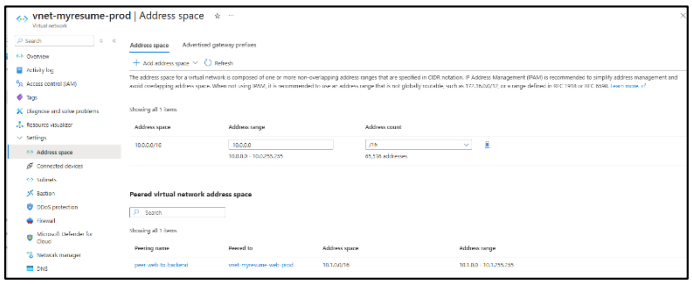
# Azure Resource & Network Provisioning

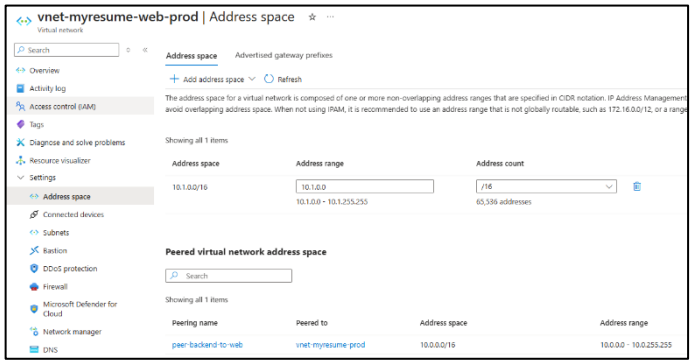
The list of resources provisioned are in 2 Resource Groups:

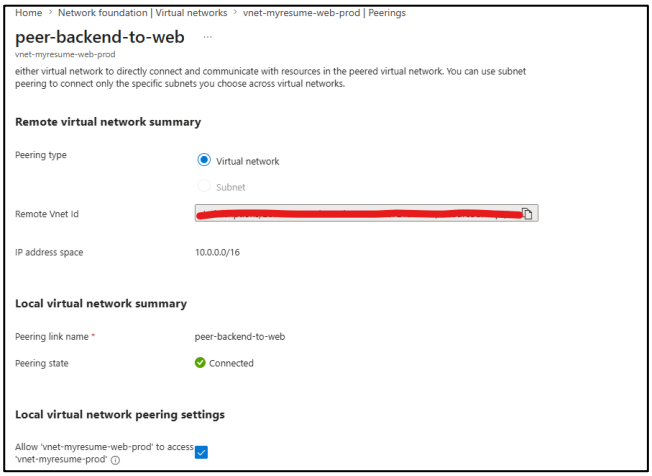
- rg-myresume-prod (Region A):
  - vnet-myresume-prod
  - lb-myresume-app
  - nat-myresume-egress
  - vm-myresume-app-01 (with nsg, disks)
  - vm-myresume-db-01 (with nsg, disks)
- rg-myresume-web-prod (Region B):
  - vnet-myresume-web-prod
  - lb-myresume-web
  - vm-myresume-web-01 (with public IP, nsg, disks)



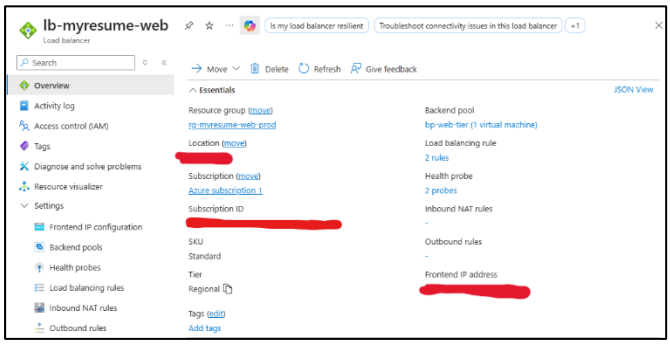
Since the Web Tier is in a different region from the rest of the resources, I had to configure VNet Peering so that vnet-myresume-prod is able to communicate with vnet-myresume-web-prod.



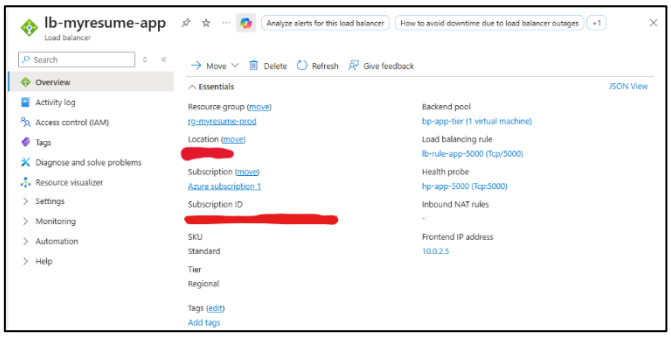



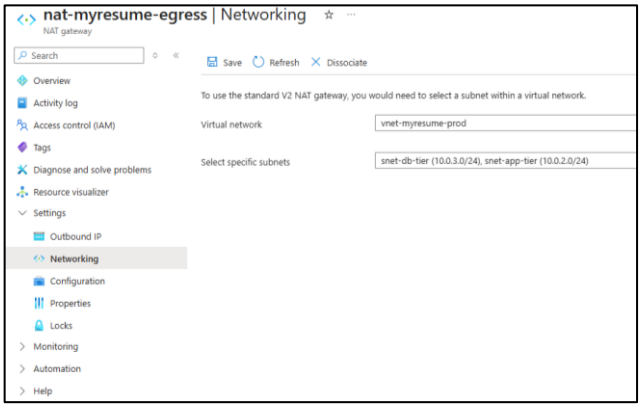

The Public Load Balancer is configured to distribute incoming traffic to all instances of the backend pool bp-web-tier, consisting of vm-myresume-web-01, for HTTP 80 and HTTPS 443




The Internal Load Balancer is configured to receives incoming traffic from the web tier and distributes the traffic to all instances of the backend pool, bp-app-tier, consisting of vm-myresume-app-01, for port 5000 (where the backend service runs)

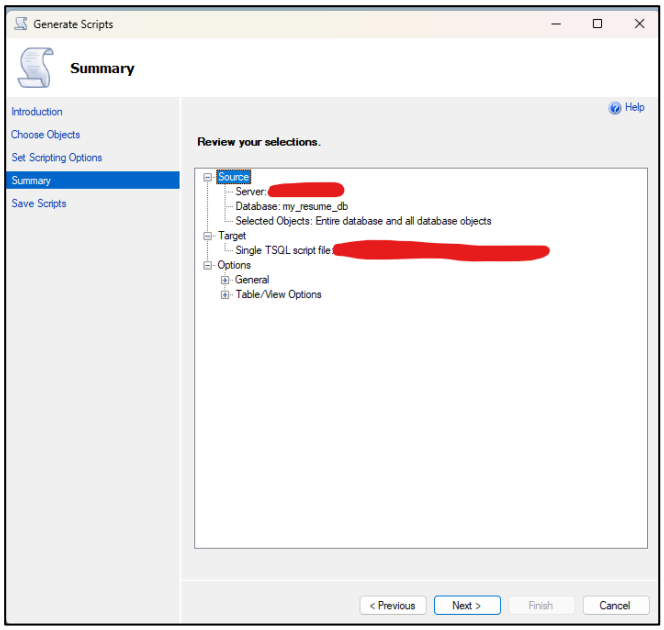


A NAT Gateway is configured for the app and db subnets so that the VMs are able to connect to Microsoft Packages to install .NET Core Runtime 10 (app) and mssql (db)



# The Data Tier: Database Configuration

From the local database, a migration script was generated to move the database to the db server.




After getting the mssql package from Microsoft Packages, mssql is installed in the db vm followed by the Migration Script execution to bring over live data to mssql

```
azureuser@vm-myresume-db-01:~$ systemctl status mssql-server --no-pager
● mssql-server.service - Microsoft SQL Server Database Engine
     Loaded: loaded (/usr/lib/systemd/system/mssql-server.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-05-22 14:53:06 UTC; 7s ago
       Docs: https://docs.microsoft.com/en-us/sql/linux
   Main PID: 2476 (sqlservr)
      Tasks: 81
     Memory: 407.2M (peak: 407.5M)
        CPU: 8.403s
     CGroup: /system.slice/mssql-server.service
             ├─2476 /opt/mssql/bin/sqlservr
             └─2524 /opt/mssql/bin/sqlservr
```

# The Application Tier: .NET Core Web API

I first had to install .NET 10 runtime on the app server since it is required to execute compiled .NET applications

A service was created for the application, myservice-api.service. It is then registered and started.

```
[Unit]
Description=MyResume .NET Core Backend API
After=network.target

[Service]
WorkingDirectory=/home/azureuser/myresume-api
ExecStart=/usr/bin/dotnet /home/azureuser/myresume-api/MyResumeBackend.dll --urls "http://0.0.0.0:5000"
Restart=always
# Restart service after 10 seconds if the dotnet service crashes:
RestartSec=10
KillSignal=SIGINT
SyslogIdentifier=myresume-api
User=azureuser
Environment=ASPNETCORE_ENVIRONMENT=Production
Environment=DOTNET_PRINT_TELEMETRY_MESSAGE=false
Environment=ASPNETCORE_URLS=http://0.0.0.0:5000

# Secure Environment Variables
Environment=ConnectionStrings__DefaultConnection="Server=10.0.3.4;Database=my_resume_db;User Id=sa;Password=<Password>;TrustServerCertificate=True;"
Environment=Cors__AllowedOrigins="<Public IP>, <domain.com>, <www.domain.com>"

[Install]
WantedBy=multi-user.target
```

The compiled .NET binaries were deployed to /var/www/myresume-api via secure copy protocol (SCP) and managed via a systemd unit service

```
azureuser@vm-myresume-app-01:~$ systemctl status myresume-api.service
● myresume-api.service - MyResume .NET Core Backend API
     Loaded: loaded (/etc/systemd/system/myresume-api.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-05-23 16:09:35 UTC; 2min 31s ago
   Main PID: 779 (dotnet)
      Tasks: 19 (limit: 9492)
     Memory: 99.9M (peak: 102.0M)
        CPU: 1.117s
     CGroup: /system.slice/myresume-api.service
             └─779 /usr/bin/dotnet /home/azureuser/myresume-api/MyResumeBackend.dll --urls http://0.0.0.0:5000
```

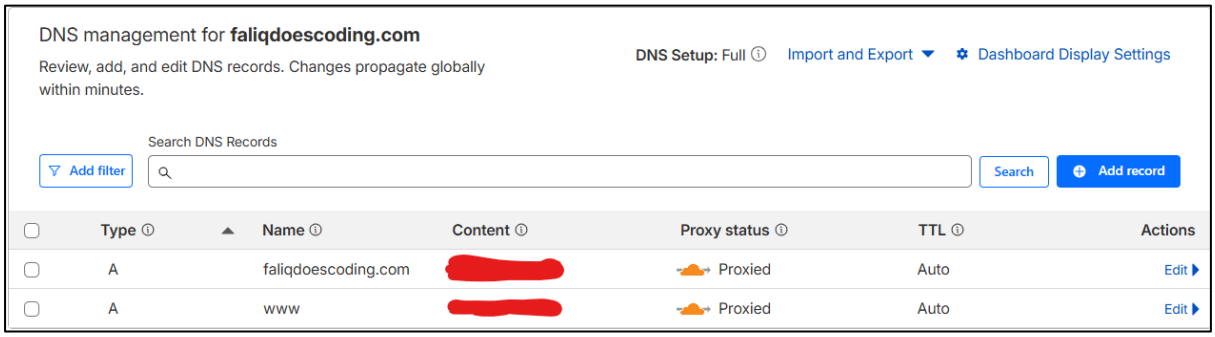
# The Web Tier: React Typescript & Nginx Reverse Proxy

Nginx was first installed in the web vm. Nginx will be responsible to serve the static compiled React TS assets directly and acting as a reverse proxy for API traffic. This means that Nginx will act as the middleman between the external users and internal backend servers. Nginx will receive the incoming requests from the internet and “proxy” (forward) them to the correct internal service. This way, it hides the IP of the backend servers.

```
server {
    server_name <domain.com> <www.domain.com> localhost;

    # 1. React Frontend Assets
    location / {
        root /var/www/myresume;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }

    # 2. .NET Core Backend API Proxy (Sanitized & Corrected)
    location /api/ {
        proxy_pass <Internal Load Balancer Front IP>;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-Port 443;
    }

    # 3. SSL Configuration
    listen [::]:443 ssl ipv6only=on;
    listen 443 ssl; # managed by Certbot
    ssl_certificate <Fullchain PEM File>;
    ssl_certificate_key <Private PEM file>;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
}

# 4. Global HTTP to HTTPS Redirector
server {
    if ($host = <www.domain.com>) {
        return 301 https://$host$request_uri;
    }

    if ($host = <domain.com>) {
        return 301 https://$host$request_uri;
    }

    server_name <Public IP Address> <domain.com> <www.domain.com>;
    return 404;
}
```

Certbot in Nginx used the domain name that I bought from Cloudflare to perform a cryptographic security check and issue an SSL/TLS security certificate once proven.

DNS resolution for faliqdoescoding.com is mapped to the front IP address of the Public Load Balancer in Cloudflare. Best security practice settings are also configured there.



# Conclusion

This phase established the core competency of building a traditional 3-tier architecture, providing a hands-on understanding of how frontends, backend APIs and relational databases interact. It served as the essential starting point for mastering manual environment management and service integration with scripts.